

Maximum flow - MPM algorithm

MPM (Malhotra, Pramodh-Kumar and Maheshwari) algorithm solves the maximum flow problem in $O(V^3)$. This algorithm is similar to [Dinic's algorithm](#).

Algorithm

Like Dinic's algorithm, MPM runs in phases, during each phase we find the blocking flow in the layered network of the residual network of G . The main difference from Dinic's is how we find the blocking flow. Consider the layered network L . For each node we define its *inner potential* and *outer potential* as:

$$p_{in}(v) = \sum_{(u,v) \in L} (c(u,v) - f(u,v)) \quad p_{out}(v) = \sum_{(v,u) \in L} (c(v,u) - f(v,u))$$

Also we set $p_{in}(s) = p_{out}(t) = \infty$. Given p_{in} and p_{out} we define the *potential* as $p(v) = \min(p_{in}(v), p_{out}(v))$. We call a node r a *reference node* if $p(r) = \min\{p(v)\}$. Consider a reference node r . We claim that the flow can be increased by $p(r)$ in such a way that $p(r)$ becomes 0 . It is true because L is acyclic, so we can push the flow out of r by outgoing edges and it will reach t because each node has enough outer potential to push the flow out when it reaches it. Similarly, we can pull the flow from s . The construction of the blocked flow is based on this fact. On each iteration we find a reference node and push the flow from s to t through r . This process can be simulated by BFS. All completely saturated arcs can be deleted from L as they won't be used later in this phase anyway. Likewise, all the nodes different from s and t without outgoing or incoming arcs can be deleted.

Each phase works in $O(V^2)$ because there are at most V iterations (because at least the chosen reference node is deleted), and on each iteration we delete all the edges we passed through except at most V . Summing, we get $O(V^2 + E) = O(V^2)$. Since there are less than V phases (see the proof [here](#)), MPM works in $O(V^3)$ total.

Implementation

```
struct MPM{
    struct FlowEdge{
        int v, u;
```